

Формат протокола JSON-RPC

Общие сведения

Соответствие стандарту JSON-RPC v2.0

Протокол передачи между клиентом и сервером, использующий JSON-формат, поддерживает спецификацию JSON-RPC v2.0 и соответствует данному стандарту. Все дополнительные типы данных описываются отдельно, дополняя существующие в спецификации типы данных.

Объект запроса в JSON-RPC v2.0

Согласно стандарту JSON-объект запроса должен содержать:

- значение **"jsonrpc": "2.0"**, указывающее на версию спецификации JSON-RPC;
- параметр **method** со значением метода бизнес-логики;
- параметр **params** с объектом передаваемых параметров;
- параметр **id** - идентификатор (устанавливается клиентом), который должен содержать строку, число или значение NULL, если включено.

Также JSON-объект запроса для использования в СБИС 3 должен содержать значение **protocol** – целое значение версии протокола представления дополнительных внутренних типов СБИС 3 в виде JSON-объектов. В настоящее время все запросы с веб-интерфейса маркируются **"protocol": 7**.

Объект ответа в JSON-RPC v2.0

Согласно стандарту JSON-объект ответа должен содержать значение **"jsonrpc": "2.0"**, указывающее на версию спецификации JSON-RPC, а также параметр **id** со строковым значением, аналогично объекту запроса. Также в ответе присылается **protocol** – версия представления внутренних данных JSON-RPC внутренних типов СБИС 3.

Если произошла ошибка, в JSON-объекте ответа должно появиться значение **error**. Описание объекта см. [Объект ошибки в JSON-RPC v2.0](#).

Если метод выполнен успешно, в JSON-объекте ответа появится значение **result** со значением, которое вернул метод. В отличие от **error**, значение **result** не имеет чёткого формата и зависит от того, что вернул метод. Например, если метод вернул null, то в JSON-объекте ответа появится **"result": null**.

Пример

```
1 {"jsonrpc": "2.0", "result": true, "id": null} // Boolean
2 {"jsonrpc": "2.0", "result": "Привет", "id": null} // String
```

Объект ошибки в JSON-RPC v2.0

Объект **error** содержит сведения об ошибке. Поля объекта:

- **code** – целочисленный код ошибки;
- **message** – текстовое сообщение об ошибке;
- **data** – в данный момент используется в СБИС 3 для передачи информации о типе исключения и **traceback** места возникновения исключения с сервера на клиент.
 - **classid** – uuid класса исключений. Подробнее об установке исключений написано [здесь](#).
 - **addinfo** – дополнительные данные для класса исключений. В зависимости от classid содержат разные данные произвольной структуры. В большинстве случаев содержит значение null.
 - **error_code** – код исключения.
 - Значение -1 свидетельствует, что исключение не задано. Т.е. произошло исключение, для которого отдельного кода не предусмотрели.

- Значение `error_code` уникально только в рамках одного `classid`. Например, значения `error_code` для `classid1` и `classid2` могут совпадать, но иметь разный смысл. Поэтому в целях отладки следует смотреть на пару `classid` + `error_code`.

- **details** — детальное сообщение об ошибке, предназначенное для упрощения отладки.

Не показывается пользователю в сообщении, но перед отправкой логируется именно это детальное сообщение с подробным описанием причины проблемы.

- **user_hint** — дополнительная подсказка в пользовательском интерфейсе.
- **type** — содержит либо "error", либо "warning" и влияет на запись в логи.

Сериализация типов в JSON-формат

Скалярные типы в JSON-RPC v2.0

Все типы скаляров представляются, согласно спецификации JSON-RPC v2.0:

- целые числа – в десятичном представлении, например: `-10`, `5`, `0`, `100`;
- вещественные числа – в виде десятичной дроби, например: `3.14`, `1.024`, `3.333`;
- текстовые строки – в обрамлении кавычками, например: `"строка"`, `"текст"`;
- логические значения – истинность в виде `true` или `false`, без кавычек;
- пустое значение – значение `null`, без кавычек.

Массивы и объекты в JSON-RPC v2.0

Все массивы представляются в виде перечисления элементов в квадратных скобках через запятую, например: `[]` – пустой массив, `["a","b","c"]` – массив строк или `[1.0,"строка",null,{"key":123}]` – массив разнородных элементов.

Все объекты представляются в виде перечисления пар **ключ:значение**, где **ключ** – строковая константа, а значением может быть произвольное значение любого типа, например: `{}` – пустой объект, `{"a":null,"b":123,"c":[false,true]}` – объект с разнородными значениями.

RecordSet – выборка данных в JSON-RPC для СБИС 3

Чаще всего результат может возвращаться в виде таблицы значений. Такой результат называется выборкой или списком записей. Ниже дается описание выборки для `"protocol":7` – 7-й версии протокола:

```
1  {
2      "s": [<формат выборки в виде массива форматов полей>],
3      "tp": "имя формата записи",
4      "f": <идентификатор формата>, // целое число, начиная с единицы
5      "d": [<массив записей>],
6      "n": <результат постраничной навигации>,
7      "p": <путь к узлу иерархии>,
8      "m": <метаданные RecordSet-a>,
9      "_type": "recordset"
10 }
```

Атрибуты "m", "n", "s", "tp" и "p" являются опциональными, а "f", "d" и "_type" – обязательными.

RecordSet – это набор записей, которые по формату на верхнем уровне одинаковы. Если сказано, что у первой записи первое поле – целое число, второе – строка, а третье – денежный формат, то у всех записей будет так.

Допустим, записи некоторого RecordSet'a содержат первое поле – целое число, второе – строка, а третье – вложенный Record. Однако такие Record'ы между собой могут различаться, например, вложенная запись первой записи RecordSet'a может иметь формат, состоящий из одного целочисленного поля, а вложенная запись второй записи RecordSet'a – формат, состоящий из одного поля типа RpcFile. То же самое касается вложенных полей типа RecordSet. Соответственно, для каждого вложенного поля Record'a или RecordSet'a необходимо сериализовать их формат.

Например, у нас есть внешний RecordSet, содержащий 100 записей и в формате этого RecordSet'a есть поле с типом вложенный Record. Это значит, что для каждой записи этого RecordSet'a мы должны записать формат его вложенного Record'a, т.е. всего мы запишем 100 форматов. Однако на практике, как правило, форматы вложенных записей в своем большинстве могут быть одинаковыми или иметь небольшое количество разных видов. Соответственно, возникает идея оптимизации процедуры сериализации за счет сокращения числа записываемых форматов.

Оптимизация состоит в том, что мы по ходу сериализации самого внешнего RecordSet'a каждому формату, который встречается впервые, присваиваем номер, начиная с единицы ("f":1). Далее, если мы видим, что формат уже ранее встречался, то указывается только атрибут "f", а "s" (описание формата) не указывается.

В старых версиях ("protocol":1 и 2) формат выборки представлялся в виде JSON-объекта. Начиная с 3-ей версии имя поля указывается в структуре формата поля. Таким образом, появляется строгий порядок полей в формате.

О формате каждого типа поля будет подробно написано ниже, в целом, поля – это простые JSON-объекты такого типа:

```
{ "n": "@Документ", "t": "Число целое" }
```

Пример сериализованной выборки в версии "protocol":7 протокола JSON-RPC v2.0:

```
1  {
2    "jsonrpc": "2.0",
3    "result": {
4      "s": [
5        {
6          "n": "@СвязиПользователя",
7          "t": "Число целое"
8        },
9        {
10         "n": "СтруктураПредприятия",
11         "t": { "n": "Связь", "t": "СтруктураПредприятия" }
12       },
13       {
14         "n": "ЧастноеЛицо",
15         "t": { "n": "Связь", "t": "ЧастноеЛицо" }
16       },
17       {
18         "n": "Пользователь",
19         "t": { "n": "Связь", "t": "Пользователь" }
20       },
21       {
22         "n": "Настройки",
23         "t": "Запись"
24       }
25     ],
26     "d": [
27       [2965, 14467304, 19544, 3088,
28         { "s": [
29           {
30             "n": "Описание",
31             "t": "Строка"
32           }
33         ]},
34       "d": ["Описание первой записи"],
35       "f": 2,
36       "_type": "record" }],
37       [2966, 14467305, 19545, 3089,
38         { "d": ["Описание второй записи"],
39         "f": 2,
40         "_type": "record" } ]
41     ],
42     "f": 1,
43     "_type": "recordset" }
```

```

44     },
45     "id":1,
46     "protocol":7
47 }

```

Record – запись в выборке данных в JSON-RPC для СБИС 3

Механика назначения формата для Record аналогична RecordSet, но отсутствуют атрибуты "m", "n" и "p".

Запись в версиях до "protocol":3 и старше сильно отличается. В версии "protocol":7 запись в JSON выглядит так:

```

1  {
2      "s":<массив форматов полей>,
3      "d":<массив значений>,
4      "f":<идентификатор формата>,
5      "tp":<имя формата>,
6      "_type":"record"
7  }

```

А в версиях "protocol":1 и 2 запись в JSON выглядит так:

```

{ "s":<объект вида {<имя поля>:<формат поля>}>, "d":<объект вида {<имя поля>:
<значение>}>}

```

Сериализация форматов типов СБИС 3

Схема сериализации форматов

Ниже представлено описание формата на примере JSON в виде нотации Бэкуса-Наура.

Начиная с релиза платформы 3.0.15 в объект передачи данных в формате JSON введена версия протокола. Для этого в главном объекте введен параметр "protocol":N. По умолчанию если протокол не указан, подразумевается, что N=1.

Первая версия протокола используется для совместимости новой версии платформы с форматом передачи данных, поддерживаемым в СБИС 2. Во второй версии протокола введен новый тип данных "Идентификатор", который может содержать первичный ключ объекта и имя объекта, а также изменен тип "Иерархия", который поддерживает сложные списки, содержащие в себе "Идентификаторы". Веб-движок платформы версии >= 3.0.15 автоматически проставляет версию протокола >=2.

Элементы формата JSON, совпадающие с элементами нотации БНФ выделены жирным:

```

1  <выборка> := { ["s":<формат выборки>[, "d":<данные>[, "n":<навигация>],
2      "tp":<имя формата>][, "k":<имя поля>]] }
3  <формат выборки>:= [<поле>[, <поле>]*]
4  <поле>:= {
5      "n":<имя поля>,
6      "t":<тип поля>
7  }
8  <имя поля>:=<строка>
9  <тип поля>:= <тип СБИС++ простой>|<тип СБИС++ сложный>
10 <тип СБИС++ простой>:= "Число целое" | "Число вещественное" | "Деньги"
    | "Дата" |
11 "Дата и время" | "Время" | "Логическое" | "Символ" | "Строка" | "Текст"
    | "Двоичное" | "Файл" | "UUID" | "Идентификатор"
12 <тип СБИС++ сложный>:= {
13     "n": "Перечисляемое"|"Флаги",
14     "s":<НаборФлагов>
15 } | {
16     "n": "Связь",
17     "t":<таблица СБИС++>

```

```

17 } | {
18     "n": "Выборка",
19     "s":<формат выборки>
20 } | {
21     "n": "Иерархия",
22     "b":<булевское>[, "f":<строка>]
23 }
24 <таблица СБИС++> := <строка>
25 <НаборФлагов> := { [<Флаг>[, <Флаг>]*] }
26 <Флаг> := <КлючФлага> <ОписаниеФлага>
27 <КлючФлага> := <целое число>:
28 <ОписаниеФлага> := <строка>
29 <Маска флагов> = [[<НомераМасокФлагов>],[<ЗначенияМасокФлагов>],
    <ОперацияМаски>]
30 <НомераМасокФлагов> := [ [-] <КлючФлага> [,-] <КлючФлага> ]* ]
31 <ЗначенияМасокФлагов> := [<булевское> [,-] <булевское> ]* ]
32 <ОперацияМаски> := <булевское>
33 <данные>:= [ [<запись>[, <запись>]*] ]
34 <запись>:=[ [<элемент записи>[, <элемент записи>]*] ]
35 <элемент записи> := <строка> | <число> | <данные> | <иерархия>
36 <иерархия> := [ <Идентификатор>, <ПризнакУзла> ]
37 <Идентификатор> := [ <целоечисло>[, <ИмяОбъекта>] ]
38 <ИмяОбъекта> := <строка>
39 <ПризнакУзла> := <булевское> | null
40 <ИдентификаторЗаписиРодителя> := <целоечисло>
41 <навигация> := <булевское> | <целоечисло>

```